

M15 - Azure / Microsoft Fabric for Data Engineering

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Primary teacher

The lesson explains from zero and gives a concrete case before asking for proof. Videos/docs are reinforcement, not a rescue requirement.

Contents

- DE15-01** - Cloud fundamentals for data engineers
- DE15-02** - Identity, IAM and least privilege
- DE15-03** - Azure Storage concepts and OneLake
- DE15-04** - Fabric workspace and item model
- DE15-05** - OneLake paths, files, tables and shortcuts
- DE15-06** - Fabric Lakehouse and Delta tables
- DE15-07** - Fabric notebooks
- DE15-08** - Spark in Fabric
- DE15-09** - Fabric Data Pipelines
- DE15-10** - SQL analytics endpoint and warehouse choice
- DE15-11** - Medallion architecture in Fabric
- DE15-12** - Incremental loads and MERGE in cloud
- DE15-13** - Permissions, connections and secrets
- DE15-14** - Monitoring, reliability and cost awareness
- DE15-15** - Git, deployment and environment separation
- DE15-16** - End-to-end Fabric data engineering case

DE15-01 - Cloud fundamentals for data engineers

What you will be able to do

Explain cloud shared-responsibility, elasticity, regions/services and how a data-engineering workload maps to managed compute/storage.

Why this matters

Cloud moves responsibility boundaries; it does not make architecture/quality/security disappear.

Picture it this way

Instead of owning the whole power plant/building, you rent managed rooms and utilities, but you still decide what data enters, who gets keys and how much electricity you consume.

Start from zero

- Cloud provider operates physical infrastructure and managed service layers according to service model; customer still owns data/config/identity/workload correctness.
- Regions provide geographic deployment boundaries with latency/residency/availability implications.
- Managed services reduce server administration but introduce service limits, identity and cost governance.
- Elastic compute can scale but costs scale too; shut down/size deliberately.
- Cloud storage/compute/network are separate cost/failure/security dimensions.
- Design should remain portable conceptually: object storage, compute, orchestration, SQL serving, IAM, monitoring.

Teacher demonstration / case

```
# Decision worksheet (supplied M15_G01_CLOUD_DECISION.md)
Requirement: daily 5 GB files, 2-hour SLA, BI serving, sensitive customer data
Decide:
- storage service / logical lake
- transform compute
- orchestration
- serving
- identity/security
- cost controls
```

Read it like English

- The exercise is architecture reasoning, not clicking Fabric first.
- Every selected managed service must map to a requirement and responsibility.

Expected check

question	evidence
who patches service host?	provider/managed boundary
who validates data?	your pipeline/team
who controls IAM/data access?	shared/customer config
cost	usage/capacity responsibility

Common beginner traps

- Calling cloud infinitely scalable/cheap.
- Assuming provider encrypts/manages every security decision for you.
- Choosing services before workload/SLA/data sensitivity.

Now you do a similar task

Complete the cloud decision worksheet for the supplied case and write three responsibilities that remain yours even with managed Fabric.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What production problem can cloud management reduce but never remove from a data pipeline?

Answer in your own words before continuing.

DE15-02 - Identity, IAM and least privilege

What you will be able to do

Use identities/roles and least privilege instead of broad admin/share-everything access.

Why this matters

Cloud data platforms centralize valuable data; an over-privileged identity can expose or modify far more than one local file.

Picture it this way

IAM is the keycard system: give each person/service only the doors/actions needed for the job.

Start from zero

- Authentication proves identity; authorization decides permitted actions.
- Human users and workload/service identities should be distinguishable.
- Least privilege: grant only needed workspace/item/data actions, for the needed scope/duration.
- Avoid sharing owner/admin credentials with notebooks/pipelines.
- Separate dev/test/prod identities/permissions.
- Audit permissions regularly and remove stale access.

Teacher demonstration / case

```
# Role reasoning
pipeline_identity:
  read: landing source
  write: bronze/silver target
  no: workspace admin / delete unrelated items
bi_reader:
  read: curated Gold/SQL serving
  no: raw sensitive landing write
```

Read it like English

- The exact Fabric role names/UI can evolve; the principle is scope/action minimization.
- The supplied security exercises focus on permission intent before UI clicks.

Expected check

identity	minimum need
pipeline	source read + target write
BI reader	serving read
developer	dev workspace authoring
production admin	limited trusted operators

Common beginner traps

- Giving every collaborator workspace admin.
- Using one shared user credential for automated pipeline.
- Least privilege only at workspace but unrestricted sensitive data elsewhere.

Now you do a similar task

Design an access matrix for developer, pipeline identity, BI reader and operator. Mark read/write/admin per layer/item and defend every grant.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why should a pipeline normally not run as the same high-privilege human administrator account?

Answer in your own words before continuing.

DE15-03 - Azure Storage concepts and OneLake

What you will be able to do

Understand object/data-lake storage concepts and OneLake as Fabric's logical unified data lake boundary.

Why this matters

Fabric items often store/access data through OneLake. Paths and ownership still matter for lineage, duplication and security.

Picture it this way

OneLake is the campus storage system: many Fabric items have logical rooms/folders in one shared lake namespace.

Start from zero

- Cloud object/lake storage organizes data by logical paths/objects, not local disk assumptions.
- OneLake provides a unified logical data lake for Fabric items/workspaces.
- Separate Files-style unstructured/raw areas from managed Tables/Delta semantics.
- Path naming should encode stable layer/domain/date meaning rather than personal usernames.
- Access to workspace/item/data remains governed; unified lake does not mean everyone can read everything.
- Shortcuts can reference data without physically copying it, changing ownership/latency/dependency reasoning.

Teacher demonstration / case

```
# Conceptual map
OneLake/
  workspace /
    lakehouse /
      Files/bronze/orders/...
      Tables/silver_orders
      Tables/gold_daily_sales
```

Read it like English

- Files and Tables have different table-management expectations.
- A path is part of reproducibility/lineage; notebook hardcoded personal paths are fragile.

Expected check

concept	meaning
OneLake	Fabric unified logical lake
Files	file/object area
Tables	managed lakehouse table area
shortcut	logical reference to external/other data

Common beginner traps

- Treating OneLake as local C: drive.
- Hardcoding personal workspace IDs/paths throughout code.
- Copying data when a shortcut/reference would satisfy ownership/freshness needs.

Now you do a similar task

Use M15_G03_ONELAKE_MAP.md to draw source -> OneLake Files/Tables paths and state which team owns each.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What changes operationally when you use a shortcut instead of copying the data into your lakehouse?

Answer in your own words before continuing.

DE15-04 - Fabric workspace and item model

What you will be able to do

Navigate Fabric conceptually as workspaces containing items with different responsibilities rather than one giant notebook area.

Why this matters

Workspace/item boundaries influence deployment, access, ownership and environment separation.

Picture it this way

A workspace is a project building; lakehouse, notebook, pipeline and warehouse are specialized rooms/tools inside it.

Start from zero

- A workspace groups Fabric items and is a collaboration/security/deployment boundary.
- Items include Lakehouse, Warehouse, Notebook, Data Pipeline and other experiences that evolve over time.
- Use item type based on responsibility: orchestration vs code vs storage/table vs SQL serving.
- Name items by domain/environment/function, not Untitled Notebook 17.
- Track dependencies between items for deployment/lineage.
- Do not put dev and prod work in the same uncontrolled item/workspace merely for convenience.

Teacher demonstration / case

```
# Supplied mapping asset
M15_G02_WORKSPACE_ITEMS.json
# Inspect item names/types/responsibilities and propose dev/test/prod naming.
```

Read it like English

- The JSON asset lets you practise without Fabric access.
- You should be able to explain why each item exists before creating it in UI.

Expected check

item	role
Lakehouse	lake files/Delta tables
Notebook	code/Spark transformation
Pipeline	orchestration/copy/control flow
Warehouse/SQL endpoint	SQL serving depending need

Common beginner traps

- Creating every service/item "just in case".
- One workspace for personal dev and production.
- Notebook used as scheduler + secrets store + storage contract all at once.

Now you do a similar task

Inspect workspace-items asset and produce an architecture map with owner/environment and dependencies.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is a Fabric workspace more than a folder for organizing icons?

Answer in your own words before continuing.

DE15-05 - OneLake paths, files, tables and shortcuts

What you will be able to do

Choose between OneLake files, managed tables and shortcuts based on ownership, copy cost and table semantics.

Why this matters

The same data can be physically copied, registered as table or referenced. Wrong choice causes duplication/staleness or missing transactional semantics.

Picture it this way

You can photocopy a book, place the original on a managed shelf, or put a pointer card to another library. Each has different ownership and freshness.

Start from zero

- Files are appropriate for raw/landing/artifacts without table semantics.
- Lakehouse Tables use Delta/table semantics for structured data.
- Shortcut references data in another supported location logically without full copy.
- Copy creates independent bytes and lifecycle; shortcut retains dependency on source availability/permissions/schema.
- Do not treat a shortcut source as data you control for retention/backfill unless contract says so.
- Document path/table/shortcut source in lineage.

Teacher demonstration / case

Decision:
raw vendor CSV -> Files/bronze landing copy (preserve evidence)
validated orders -> Tables/silver_orders
shared reference master already governed elsewhere -> consider shortcut
Gold aggregate -> managed table/warehouse serving

Read it like English

- Raw evidence may justify copying even when shortcut exists.
- Reference data may benefit from no-copy shortcut if ownership/SLA acceptable.

Expected check

choice	main tradeoff
copy	independent data + storage/refresh
shortcut	no copy + source dependency
table	structured transactional/query semantics

Common beginner traps

- Shortcut for volatile source then assuming historical immutability.
- Copying large shared data into every workspace.
- Storing curated tables only as CSV files.

Now you do a similar task

For three datasets (raw daily vendor file, shared customer master, curated sales fact), choose copy/files/table/shortcut and defend ownership/replay needs.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why might raw ingestion intentionally copy data even if a shortcut could reference the source?

Answer in your own words before continuing.

DE15-06 - Fabric Lakehouse and Delta tables

What you will be able to do

Use a Fabric Lakehouse as OneLake storage plus managed Delta tables, carrying forward Delta correctness rules.

Why this matters

Fabric simplifies lakehouse management but Delta grain/schema/MERGE/replay rules remain your responsibility.

Picture it this way

Fabric provides the managed building and catalog; Delta table is still the ledger-backed shelf whose key/grain you must design.

Start from zero

- Fabric Lakehouse combines Files and managed Tables backed by OneLake/Delta.
- Bronze raw can live in Files or Delta depending contract; Silver/Gold structured data commonly use Delta tables.
- Delta transaction/history/schema enforcement concepts from M13 carry over.
- Do not assume Fabric table automatically has correct business key/quality.
- SQL analytics access may expose Lakehouse tables for querying; write capabilities differ by experience/tool and evolve.
- Validate row/key/measure controls after notebook/pipeline writes.

Teacher demonstration / case

```
# Offline simulator/lab illustrates Lakehouse I/O contract
python M15_G04_LAKEHOUSE_IO.py
# Inspect input -> bronze/silver/gold artifacts and control totals.
```

Read it like English

- The offline asset reproduces layer/path/table reasoning without paid capacity.
- When you have Fabric access, map the same logic to Lakehouse items rather than relearning concepts.

Expected check

Fabric object	data-engineering rule
Lakehouse table	Delta grain/key/schema
Files	raw/object contract
SQL endpoint	query/serving access, not quality guarantee

Common beginner traps

- Thinking managed service removes Delta validation.
- Writing raw/clean/gold all into same table.
- Assuming local simulator proves Fabric UI/runtime.

Now you do a similar task

Run the offline Lakehouse I/O lab and write how each local path maps to a Fabric Lakehouse Files/Tables item.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Which Delta concepts remain necessary even when Fabric creates the table for you?

Answer in your own words before continuing.

DE15-07 - Fabric notebooks

What you will be able to do

Use notebooks as versionable/reproducible transformation interfaces rather than manual one-off interactive scratchpads.

Why this matters

Cloud notebooks are convenient but hidden cell state/manual ordering can create results nobody can reproduce.

Picture it this way

A notebook is a lab notebook: every experiment step must be repeatable from a clean kernel, not depend on an invisible variable from yesterday.

Start from zero

- Keep setup/config/input parameters near the top and avoid hidden interactive state.
- Split reusable business logic into modules/functions when it grows.
- Run cells top-to-bottom from a clean session before handoff.
- Use environment/config/secret references, not pasted credentials.
- Persist durable results to tables/files; notebook memory is temporary.
- Record run parameters/input versions and validation outputs.

Teacher demonstration / case

```
# Reproducible notebook pattern
PARAM_SOURCE_DATE = "2026-08-01"
# 1 config/session
# 2 read exact source/window
# 3 validate/transform functions
# 4 write target idempotently
# 5 reconciliation cells
# 6 save run evidence
```

Read it like English

- The source date is explicit rather than `datetime.now()`.
- Validation is a first-class final step, not just `display()`.

Expected check

notebook habit	proof
clean run	restart/run all succeeds
params	explicit
secrets	referenced, not pasted
outputs	durable table + controls

Common beginner traps

- Running cells out of order and saving result screenshots.
- Huge business logic only in unnamed cells.
- Pasting SAS/token/password into notebook.

Now you do a similar task

Take M15_G05_SPARK_TRANSFORM.py/notebook plan and turn it into a six-section notebook outline with explicit params and validation.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can a notebook that "works right now" still be unreproducible?

Answer in your own words before continuing.

DE15-08 - Spark in Fabric

What you will be able to do

Translate local PySpark skills to managed Fabric Spark while controlling compute, file layout and driver behavior.

Why this matters

Managed Spark removes cluster setup burden but not shuffle/skew/collect/partition mistakes or compute cost.

Picture it this way

The cloud gives you a managed factory, but bad join keys or collect() can still overload the manager and waste paid machine time.

Start from zero

- SparkSession/DataFrame concepts remain the same.
- Compute starts/stops/scales under Fabric capacity/session model; startup/runtime consumes capacity.
- Use Delta/Parquet-native operations and built-ins; avoid Python UDF/collect when unnecessary.
- Inspect plans and partition/file layout for expensive jobs.
- Do not leave interactive compute running unnecessarily when capacity/cost model charges for use.
- Validate output independently from notebook green status.

Teacher demonstration / case

```
# Same conceptual transform
completed = orders.filter("status = 'Completed'")
result = completed.groupBy("region").sum("gross_sales")
# Then reconcile total, inspect plan, write Delta table.
```

Read it like English

- The code looks familiar from M12; environment/paths/catalog differ.
- Managed execution changes operations/cost, not correctness principles.

Expected check

carry-over skill	cloud concern
shuffle/join plan	capacity/runtime cost
partitioned Delta write	OneLake file layout
collect safety	driver memory
session lifecycle	capacity use

Common beginner traps

- Assuming managed Spark makes collect safe.
- No plan/control-total evidence.
- Using huge capacity to hide inefficient join.

Now you do a similar task

Use offline Spark transform asset and write the Fabric-specific concerns you would add: item/path, capacity/session, permissions and monitoring.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What Spark performance mistake becomes a direct cost concern in a managed cloud environment?

Answer in your own words before continuing.

DE15-09 - Fabric Data Pipelines

What you will be able to do

Design Fabric Data Pipelines as orchestration/copy/control-flow boundaries with explicit parameters, retries and quality gates.

Why this matters

Cloud pipelines coordinate movement/notebooks/SQL activities but can still publish partial or duplicate data if state/idempotency is weak.

Picture it this way

A Fabric pipeline is the control tower coordinating copy/notebook/check activities; each activity still needs a correct contract.

Start from zero

- Pipeline activities can copy data, invoke notebooks/stored procedures and coordinate dependencies depending service features.
- Parameterize source dates/paths/environment instead of editing pipeline for every run.
- Put validation/reconciliation before publish/state advancement.
- Retry only transient activities and make target writes idempotent.
- Use failure branches/alerts and preserve prior serving data on deterministic failures.
- Record run/activity IDs/status/duration/counts for operations.

Teacher demonstration / case

```
# Supplied plan asset
M15_G07_PIPELINE_PLAN.json
# Review activity order:
copy/land -> validate -> transform -> quality gate -> publish -> state/evidence
```

Read it like English

- The JSON plan is intentionally inspectable offline.
- When using Fabric UI, the same activity graph/parameters should remain explainable.

Expected check

activity	responsibility
copy/land	preserve source
notebook/transform	deterministic logic
quality gate	block bad publish
publish/state	idempotent after validation

Common beginner traps

- Retrying deterministic schema failure.
- Advancing watermark before publish.
- Manual one-off pipeline parameters not recorded.

Now you do a similar task

Review pipeline_plan asset and add failure branches for schema failure and temporary source outage. Explain which retries and which does not.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why should a quality gate occur before the final serving publish/state update?

Answer in your own words before continuing.

DE15-10 - SQL analytics endpoint and warehouse choice

What you will be able to do

Choose Lakehouse SQL analytics endpoint vs Fabric Warehouse/other SQL serving based on workload/write/model/concurrency needs.

Why this matters

Fabric offers more than one SQL-facing experience. Choosing by name instead of requirement can create awkward write/governance/performance patterns.

Picture it this way

One SQL doorway may be a query view into Lakehouse Delta tables; another is a warehouse built around SQL-centric managed tables/workloads.

Start from zero

- Lakehouse SQL analytics endpoint exposes SQL query access over Lakehouse tables; exact capabilities evolve and should be checked in current product docs.
- Fabric Warehouse is a SQL-first warehouse experience for relational analytics/BI workloads.
- Choose based on data location/format, transformation tool preference, write needs, performance/concurrency and governance.
- Avoid duplicating the same Gold dataset into both unless a requirement justifies it.
- Test consumer query patterns and security, not only create item.
- Course focuses on decision reasoning; UI/capacity features can change.

Teacher demonstration / case

Decision questions:

1. Is the authoritative curated data already Delta in Lakehouse?
2. Are transformations primarily Spark/SQL?
3. Do consumers need SQL-centric warehouse features/concurrency?
4. What duplication/refresh would another serving store introduce?

Read it like English

- There is no universal "always choose Warehouse" or "always Lakehouse" answer.
- The source of truth/serving contract should remain clear.

Expected check

case	likely direction
Spark/Delta-centric lakehouse BI	Lakehouse + SQL endpoint may fit
SQL-centric warehouse modeling/serving	Warehouse may fit
same data in both	justify duplication/SLA

Common beginner traps

- Choosing based on product icon preference.
- Assuming endpoint can write everything same as Warehouse.
- Duplicating Gold with no lineage/refresh contract.

Now you do a similar task

Use P04 Lakehouse-vs-Warehouse exercise to choose for two workloads and state what evidence would change your choice.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What is the main risk of copying the same Gold data into two serving systems without a clear ownership contract?

Answer in your own words before continuing.

DE15-11 - Medallion architecture in Fabric

What you will be able to do

Implement Bronze/Silver/Gold contracts in Fabric while keeping raw replay, Delta quality and Gold consumer grain explicit.

Why this matters

Medallion architecture becomes operational in cloud only when layers have ownership, access, retention and validation boundaries.

Picture it this way

The cloud gives you larger warehouses, but receiving/inspection/finished-product rules still need labels and audits.

Start from zero

- Bronze: source-like landing + ingestion metadata/history.
- Silver: typed, deduped, conformed quality-controlled tables.
- Gold: consumer-specific facts/dimensions/aggregates.
- Use separate tables/paths and permissions where sensitivity differs.
- Reconcile every layer; Gold totals must trace to Silver population.
- Backfill/replay should rebuild Silver/Gold from Bronze without asking source again when possible.

Teacher demonstration / case

```
python M15_G08_MEDALLION_INCREMENTAL.py  
# Offline simulator exercises Bronze -> Silver -> Gold + incremental update.
```

Read it like English

- The script demonstrates layer progression and incremental logic.
- Map outputs to OneLake Files/Tables in Fabric design.

Expected check

layer	required proof
Bronze	raw count/identity
Silver	valid+reject/key checks
Gold	consumer grain + control totals
replay	same source safe

Common beginner traps

- Using Bronze as cleaned current-state only.
- No separate reject/quarantine evidence.
- Gold totals not reconciled.

Now you do a similar task

Run medallion incremental offline lab twice and map each artifact to planned Fabric items/permissions.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is Bronze especially valuable when a Silver business rule changes six months later?

Answer in your own words before continuing.

DE15-12 - Incremental loads and MERGE in cloud

What you will be able to do

Design incremental cloud loads with watermark/change key and Delta MERGE so retries/backfills are safe.

Why this matters

Cloud compute/movement cost makes full reloads expensive, but incremental state creates correctness risk if advanced too early.

Picture it this way

Same bookmark rule as local pipelines, now with cloud table/object operations and paid runs.

Start from zero

- Use stable source change key/watermark/partition/event identity.
- Persist state only after durable validated target commit.
- Use MERGE/upsert/version conditions for current-state tables.
- Late data policy may overlap recent windows or consume source CDC.
- Backfill parameters should not overwrite normal state.
- Record input/output table versions/run IDs for audit.

Teacher demonstration / case

```
# Cloud incremental invariant
checkpoint_before = X
extract rows > X
MERGE to Silver by business key/version
validate controls
checkpoint_after = max(successful_source_position)
# save after success
```

Read it like English

- The logic is identical in principle to M08/M13.
- Fabric managed services change mechanics, not state ordering.

Expected check

test	expected
first run	new/updated keys applied
same run replay	no duplicates
failure before state save	old state remains
backfill	normal checkpoint not corrupted

Common beginner traps

- Saving watermark as first activity.
- MERGE without source winner/version rule.
- Using current time rather than source change state.

Now you do a similar task

Design an incremental Fabric run for daily orders with watermark + late 3-day overlap and explain target MERGE/idempotency.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can cloud retries be expensive and still not fix a wrong watermark design?

Answer in your own words before continuing.

DE15-13 - Permissions, connections and secrets

What you will be able to do

Separate workspace/item permissions, data access, connections and secrets; never embed cloud credentials in notebooks/pipelines.

Why this matters

Cloud identities can reach broad data. Credential leakage and over-broad connections are high-impact.

Picture it this way

A pipeline references a locked credential entry/key vault/connection; the notebook should not contain the key printed in a cell.

Start from zero

- Use managed/workload identities or secured connections/secret stores where available.
- Grant source/target access to workload identity at minimum scope.
- Keep credentials out of Git/notebook code/parameters/log output.
- Separate dev/test/prod connections and data.
- A shortcut/external data connection has its own source-side authorization dependency.
- Audit sharing/workspace roles and rotate leaked secrets.

Teacher demonstration / case

```
# Bad
storage_key = "REAL-SECRET..."
# Good concept
connection_name = "orders_source_dev"
# Runtime identity/connection resolves credential outside code.
```

Read it like English

- Logical connection name can be versioned; actual secret changes by environment.
- Logs should show connection ID and error class, not credential value.

Expected check

artifact	safe content
notebook/pipeline JSON	connection reference
secret store/connection	credential
Git	no token/password
logs	no secret values

Common beginner traps

- Pasting SAS token in notebook.
- Same production connection in dev.
- Sharing admin workspace access to solve a read permission problem.

Now you do a similar task

Complete security/monitoring asset and produce a permission/secret flow diagram from user/workload identity to source/OneLake/Gold.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

If a notebook needs storage access, why is giving the developer's personal admin credential a bad shortcut?

Answer in your own words before continuing.

DE15-14 - Monitoring, reliability and cost awareness

What you will be able to do

Monitor pipeline/data/capacity health and reason about reliability and cost instead of only checking successful status.

Why this matters

Cloud workloads can be green but late, incomplete or unnecessarily expensive.

Picture it this way

Monitoring is both the factory quality board and the electricity meter.

Start from zero

- Track pipeline/activity status/duration/retries, source/target counts and freshness/latency.
- Track compute/capacity utilization/throttling and job runtime trends according to Fabric capacity tooling.
- Cost/consumption can be driven by Spark runtime, pipeline activity, Warehouse/BI usage and retained/copied data depending licensing/capacity model.
- Right-size schedules/compute and stop unnecessary interactive work.
- Alert on SLA/data-quality breach, not every harmless retry.
- Incident evidence should include run/item IDs, errors, data impact and capacity symptoms.

Teacher demonstration / case

```
# Monitoring evidence template
run_id, item, environment, start, end, status,
source_rows, target_rows, rejected_rows,
freshness_lag, retry_count, capacity_symptom, error_class
```

Read it like English

- The same run joins operational and data-quality evidence.
- Cost tuning happens after correctness and from measured usage.

Expected check

signal	why
duration/freshness	SLA
row controls	data correctness
retry/error	reliability
capacity utilization/throttle	resource/cost

Common beginner traps

- Only looking at success/failure badge.
- Increasing capacity before diagnosing skew/bug.
- Turning off monitoring because it costs effort.

Now you do a similar task

Use M15_G09_SECURITY_MONITORING.md to design three alerts and one cost review for the case.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

How can a successful pipeline run still represent a production incident?

Answer in your own words before continuing.

DE15-15 - Git, deployment and environment separation

What you will be able to do

Separate dev/test/prod, version code/config and deploy Fabric items without manually editing production.

Why this matters

Manual UI-only changes create drift and make rollback/audit difficult.

Picture it this way

Treat cloud data assets like software: build in workshop, verify in test, promote a known version to production.

Start from zero

- Keep code/notebooks/pipeline definitions in supported source control/deployment process where available.
- Dev/test/prod use separate workspaces/items/connections/data with controlled promotion.
- Environment-specific values belong in parameters/config, not code forks.
- CI/tests/quality gates should run before production promotion.
- Record deployed commit/item version/config and have rollback/redeploy plan.
- Do not assume every Fabric item supports identical Git/deployment behavior; verify current capability and document manual exceptions.

Teacher demonstration / case

```
dev workspace -> tests/validation -> test workspace -> approval -> prod workspace
same code/version
+ environment config/connection mapping
+ recorded deployment evidence
```

Read it like English

- The goal is identical logic with environment-specific references.
- Manual production edits after deployment create drift that should be detected/avoided.

Expected check

environment	data/permission
dev	safe synthetic/dev data, developer write
test	production-like test, gated
prod	restricted workload/operator access

Common beginner traps

- Developing directly in prod.
- Different code branches per environment with no promotion path.
- Secrets embedded because deployment parameters were inconvenient.

Now you do a similar task

Use P09 dev/test/prod exercise to design workspace/item/connection naming and promotion/rollback evidence.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why should prod configuration differ from dev while the transformation code ideally stays the same version?

Answer in your own words before continuing.

DE15-16 - End-to-end Fabric data engineering case

What you will be able to do

Defend an end-to-end Fabric-style architecture from source to OneLake Bronze/Silver/Gold, orchestration, serving, IAM, monitoring and deployment.

Why this matters

The final cloud lesson checks whether you can map core DE principles into a managed platform without becoming dependent on UI memorization.

Picture it this way

You can now design the whole managed factory: receiving, storage, Spark workstations, scheduler, SQL storefront, identity/security and operating dashboard.

Start from zero

- Start from source/SLA/volume/security requirements, not Fabric item list.
- Choose Files/Tables/shortcuts and Lakehouse/Warehouse roles explicitly.
- Parameterize/orchestrate pipeline with quality gate + safe state.
- Use Spark/SQL transformations with Delta grain/key/replay rules.
- Use least-privilege workload identity/connections.
- Monitor data controls, freshness, failures and capacity/cost.
- Separate dev/test/prod and document current Fabric UI/runtime evidence when available.
- Be able to translate the architecture to another cloud by naming generic capabilities.

Teacher demonstration / case

```
# End-to-end runbook asset
M15_G10_END_TO_END_RUNBOOK.md
# Offline simulator + project assets prove logic even without paid Fabric.
# Live Fabric evidence, if available, is a separate runtime completion check.
```

Read it like English

- The course intentionally separates transferable cloud reasoning from vendor UI access.
- The module practical contains result_summary/pipeline plan/runtime evidence fields.

Expected check

area	proof
architecture	requirements -> justified items
data	layer/key/row controls
state	incremental/replay safe
security	least privilege/no secret in code
ops	monitoring/cost/deployment

Common beginner traps

- Listing Fabric products with no data flow/contracts.
- Claiming hands-on Fabric runtime if only offline simulator was used.
- Unable to explain same design in generic cloud terms.

Now you do a similar task

Complete module practical using offline assets and, if you have Fabric access, add live runtime evidence separately. Defend every item and one alternative architecture.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Which parts of your Fabric architecture should remain valid if the company moves to another cloud platform?

Answer in your own words before continuing.